Big Data

Unit - 2 [One Shot]

Most important topics:

1. Hadoop
   - History
   - Apache hadoop
   - HDFS
   - Components of hadoop
   - Data format, analysis using hadoop
   - Hadoop streaming, Hadoop pipes
   - Hadoop echo system
2. Map Reduce
   - Framework and basics
   - Working of Map Reduce
   - Developing Map Reduce application
   - Unit tests with MR unit
   - test data and local tests
   - anatomy of a MapReduce job run
   - failures, job scheduling
   - shuffle and sort, task execution
   - Map Reduce types, input/op formats
   - Map Reduce features, Real-world Map Reduce

Hadoop:

@brevilearning

- It is like a giant, smart storage and processing system that lets many ordinary computers works together to handle large volumes of data.

* History:

- 2003: Google released a paper on Google File System (GFS).
  ↓
  (gave the idea for HDFS)
- 2004: Google released paper on MapReduce.
- 2005: Doug Cutting and Mike Cafarella started Hadoop.
  ↓
  (it was a part of a project called Nutch (web search engine))
- 2006: Hadoop became a separate project under Apache.
- 2008: Yahoo! used Hadoop to sort 1 Tb of data in 209 seconds.
  ↓
  Hadoop became popular for big data tasks.
- 2011: Apache Hadoop 1.0.0 was released.
- 2013: Apache Hadoop 2.0 released with YARN.
- 2017: Apache Hadoop 3.0 was released.

* Apache Hadoop:

- This is the most widely used and standard version of the Hadoop software.
- It's a collection of programs and tools that work together.
- Many large companies use Apache Hadoop to manage their huge amounts of data.

@brevitlearning

* Components of Hadoop:

- Hadoop is made of several parts, not just one program.
- The two most important parts are HDFS and Map Reduce.
- These components work together like a team to handle big data tasks.
- HDFS: (Hadoop Distributed File System)

→ HDFS is the storage part of Hadoop. It's designed to ~~storage~~ store extremely large files across many computers.

→ Instead of storing one huge file on one computer, HDFS breaks the file into smaller pieces and scatters copies of these pieces across many different computers.

→ If one computer storing a part of the file breaks, the data is still safe because copies are on other computers.

→ It's built for very fast reading of large amounts of data, rather than quick access to small pieces.

→ File sizes, Block sizes:

- File in HDFS is not saved as one piece.
- It's divided into smaller fixed-size chunks called 'blocks'.
- HDFS stores three identical copies of every block.

- These copies are placed on different computers in the Hadoop cluster.

→ How HDFS stores, reads and writes files

- When you save files, HDFS breaks it into blocks, makes the required copies, and then sends these copies to different "data nodes".
- When you want to read a file, HDFS quickly fetches them from the closest available data nodes.
- Files in HDFS are typically written once and then read many times.

@brevilearning

★ Data format:

- Hadoop is incredibly flexible with the types of data it can handle.
- Structured, semi-structured and unstructured data can be handled.
- This flexibility makes it suitable for a wide range of Big Data.

★ Analysis using Hadoop:

- Instead of moving massive amounts of data to one super-powerful computer for analysis, Hadoop brings the distributed system approach for analysis.
- It sends small programs (like MapReduce jobs) to the individual computers where the data is already stored.
- This approach is called "compute proximity". It saves a huge amount of time and network traffic.

@brevilearning

★ Hadoop streaming:

- It lets you write your Map Reduce programs in any executable language you want. (e.g. java, python, perl, shell script, etc)
- Hadoop streaming allows your script to communicate with Hadoop framework.

* Hadoop pipes:

- It is specifically designed for programmers who use the C++ programming language
- It provides a C++ interface to write MapReduce programs
- It can offer performance benefits for very intensive computational tasks.

* Hadoop Ecosystem:

- It is a collection of many different open-source software tools that are built to work with Hadoop's core components (HDFS & MapReduce)
- These ecosystem tools make it much easier to perform specific big data tasks.
- They simplify writes complex operations that might require writing long programs, making Big data accessible to more users and for more purposes.
- These tools don't replace Hadoop or MapReduce, they just use them.
- Some key tools:
- Hive: Allows users to query data in Hadoop using SQL, a familiar language from traditional databases.
- Pig: Provides simpler scripting language for analyzing large datasets.
- HBase: A fast, real-time database that stores data on top of HDFS, good for quick lookups.
- Spark: A very fast engine that can process data quicker than traditional mapReduce for certain types of tasks.

@brevilearning

* Map Reduce:

- It is the brain behind processing the huge amounts of data stored in HDFS.
- It's a programming model that

simplifies writing applications for processing large datasets in a parallel and distributed way across many computers.

* Framework and basics.

- Map Reduce provides a simple way to write programs that run on a large cluster of machines.

- It handles all the complicated tasks automatically, like dividing the work, distributing it, and handling errors.

- Programmers only need to focus on writing two main functions Map and Reduce.

- This framework makes it easier to develop parallel applications for big data.

* Working of MapReduce:

Entire process is based on two core phases Map and Reduce with an automatic shuffle and sort phase in between.

@brevilearning

i) Map Phase:

- Input data is broken into smaller pieces and given to individual "Mapper" processes.

- Each mapper processes its small piece of data independently.

- Mappers perform filtering, sorting, or other simple operations.

- Their o/p is a list of temporary "key-value pairs".

ii) Shuffle and Sort phase.

- This phase happens automatically after Map phase.

- All temporary key-value pairs from all the mappers are collected.

- These pairs are then shuffled so that all values belonging to the same key are sent to the same reducer.

- These pairs are also 'sorted' by

key within each group while shuffling.

iii) Reduce phase:

- Reducer processes receive the grouped and sorted key-value pairs from the shuffle and sort phase.
- Each reducer works on a unique key and its associated list of values.
- Reducers perform aggregation, summarization, or combining operations.
- Their output is the final result of the MapReduce job.

@brevilearning

* Map Reduce Application Development:

1. Developing Map Reduce Application:

- This involves writing the actual computer code for the Map and reduce functions.
- You define the logic for what each mapper should do with its input and what each reducer should do with its grouped data.
- Often, you'll also write a "Driver" program to set up and run the mapreduce job.
- These applications are typically written in java, but can use streaming for other languages.

@brevilearning

2. Unit test with MR unit:

- MRUnit is a special tool specifically for testing MapReduce programs.
- It allows testing of functions in isolation (on your own computer) without needing a full Hadoop cluster.
- Helps you quickly find and fix bugs in your logic before deploying to a large scale system.

3. Test data and Local tests:

- To test your Map Reduce programs, you create small, simple datasets called "test data".
- "Local tests" means you run your MapReduce job on your own single computer.
- It allows you to quickly check if your program's logic is correct with a small amount of data.

* Anatomy and execution of a Map Reduce job:

1. Anatomy of a Map Reduce job Run

- This refers to the entire lifecycle of a Map Reduce job, from when you submit it until it completes.
- It involves steps like job submission, resource allocation, task execution, and collecting final results.

2. Failures:

- Map Reduce is designed to be highly fault-tolerant.
- If a computer running a "Map" or "Reduce" task crashes or stops responding, Map Reduce automatically detects it.
- It fails then re-starts that specific failed task on a different computer in the cluster.

@brevilearning

3. Job scheduling:

- When many different users or applications submit MapReduce jobs to the same Hadoop cluster, a "scheduler" decides which job gets to run when.
- Schedulers manage the allocation of cluster resources (CPU time and mem) to different jobs.
- They aim to ensure fairness among jobs and efficient use

of the cluster's capability.

4. Shuffle and Sort:

- This is the critical automatic phase that connects the Map and Reduce tasks.
- Its purpose is to gather all the intermediate o/p from the Mappers.
- It then group these o/p by key and sorts them.
- This ensures that all data related to a specific key goes to the correct Reducer for final processing.

5. Task Execution:

- This refers to the actual running of individual Map and Reduce operations.
- These "tasks" are executed by "Task Trackers" (in older hadoop) or "Node Managers" (in new hadoop with Yarn) on the various computers in the cluster.
- Each task processes a small portion of the data allocated to it.

★ MapReduce types:

@brevilearning

i) Identity MapReduce:

- Used for testing and basic filtering.
- It just pass the data as it is.

ii) Chain MapReduce:

- You can use multiple mappers and reducers in one job.
- Each mapper can do different filtering or transformations.

iii) Word count type:

- Used to count frequency of words or items in a file.

iv) Sorting type:

- Map step breaks the data

into key - value pairs.

- Reduce step sorts the data by key or value.

v) Join MapReduce.

- Used to combine data from two or more files.
- Works like SQL JOIN.

@brevitylearning

★ Input / Output Formats.

- Input Formats: Defines how map reduce reads the raw data from HDFS before starting the Map phase.
- Output Formats: Define how MapReduce writes its final results back to HDFS after the reduce phase.

★ Map Reduce Features:

- Scalability: Map Reduce can easily handle ever - growing amounts of data.
- Fault - tolerance: It can automatically recovers from failures, making it very reliable.
- Simplicity: For certain problems. MapReduce model just writes map and reduce functions.
- Parallel processing: It process data by dividing works among multiple computers at once.

★ Real - world MapReduce applications:

- Web search: used by search engines to process billions of web pages and build indexes for fast searching.
- Log analysis: Analyze huge amounts of web logs to understand user behaviour, popular pages, and errors.
- Recommendation systems: Powers systems that suggest products (like on Amazon) or movies based on your past activities.
- Data warehousing: Processing and preparing massive amounts of

business data for reporting and analytical insights.

- Scientific simulations: Used in science for processing large datasets from experiments or simulations.

Thanks for watching

@brevilearning